



ASSIGNMENT #2

COMPUTER ARCHITECTURE

ID: _____

NAME: _____

SECTION: _____

Read the Instructions Carefully

- ❖ Assignments are designed for your learning copying will lead to plagiarism.
- ❖ Your assignment marks will be calculated from your submission (20%) + Quiz (80%) W)
- ❖ You are supposed to debug each question in compilers and update memory and register as instructed.
- ❖ If you find any **ERROR** in the program comment on the error and reassemble the code. You must understand ERROR carefully.

Note:

Use RISC-V assembler given below to solve the assignment.

<https://riscv-simulator-five.vercel.app/>

for programs with functions this assembler has better GUI

<https://creatorsim.github.io/creator/#>

Assignment 2

Computer Architecture

Question 1

Consider the following **RISC-V** code and analyze the working of the **div** instruction. For each instruction, update the contents of all affected registers in both hexadecimal and decimal format. You are required to clearly understand and explain the execution of every instruction step by step.

```
addi s0,zero,25
addi s1,zero,2
```

```
div s2,s0,s1
divu s3,s0,s1
```

```
rem s4,s0,s1
remu s5,s0,s1
```

```
addi s0,zero,-25
addi s1,zero,2
```

```
div s2,s0,s1
divu s3,s0,s1
```

```
rem s4,s0,s1
remu s5,s0,s1
```

```
addi s0,zero,25
addi s1,zero,-2
```

```
div s2,s0,s1
divu s3,s0,s1
```

```
rem s4,s0,s1
remu s5,s0,s1
```

```
addi s0,zero,-25
addi s1,zero,-2
```

```
div s2,s0,s1
divu s3,s0,s1
```

```
rem s4,s0,s1
remu s5,s0,s1
```

s0	
s1	
s2	
s3	
s4	
s5	
s0	
s1	
s2	
s3	
s4	
s5	
s0	
s1	

s2	
s3	
s4	
s5	
s0	
s1	
s2	
s3	
s4	
s5	

Assignment 2

Computer Architecture

Question 2

Consider the following **RISC-V** code and analyze the working of the **mul** instructions. For each instruction, update the contents of all affected registers in both hexadecimal and decimal format. You are required to clearly understand and explain the execution of every instruction step by step.

```
addi s0,zero, 4
addi s1,zero,-5
```

```
mul s2,s0,s1
mulh s3,s0,s1
```

```
mul s2,s0,s1
mulhu s3,s0,s1
```

```
mul s2,s0,s1
mulhsu s3,s0,s1
```

```
mul s2,s1,s0
mulhsu s3,s1,s0
```

s0	
s1	
s2	
s3	
s2	
s3	
s2	
s3	
s2	
s3	

Question 3

I. Write a **RISC-V** program to evaluate and compute its result. For the given expression

60

$$x = a(b + c \times d - e)$$

```
# s0 = a = 3, s1 = b = 2, s2 = c = 4, s3 = d = 6 , s5 = e = 6
#s6 = x = ?
```

Assignment 2

Computer Architecture

II. Write a **RISC-V** program to evaluate and compute its result. For the given expression

1849

$$x = a + (b \div c \times d)^2$$

s0 = a = 3, s1 = b = 10, s2 = c = 5, s3 = d = 20 , s6 = x = ?

III. Write a **RISC-V** program to evaluate and compute its result. For the given expression

48

$$x = a + b (c - d)/e$$

s0 = a = 42, s1 = b = 2, s2 = c = 14, s3 = d = 8 , s3 = e = 2
#s6 = x = ?

Assignment 2

Computer Architecture

Question 4

Consider the following **RISC-V** code and analyze the working of the **load and shift** instructions. For each instruction, update the contents of all affected registers in both hexadecimal. Also update given memory. You are required to clearly understand and explain the execution of every instruction step by step. Also fill memory for given declaration and comment statement if any errors are found

.data memory starts at 0x02000000

```
##### Put # in data declaration if error is found
## for the errors
b1: .byte +128,-5,-128,0b01110111,0xAD,'A', 'b',
    -0b01011001,255
b2: .byte -129
b3: .byte "h"
b4: .byte 'hello'
b5: .string 'hello'
b6: .asciz "hello again"
b7: .string "hello"
.align 2    #required to align on half boundry

h1: .half 0xABCD, 2,-5,0b01010110010001111,0x12AB
h2: .half 6536
h3: .half +32768
h4: .half -32769
h5: .half 'a'
.align 4    #required to align on word boundary
w1: .word 0x12345678, +5,-5, 0xABCEF712, 0x12345DEF
```

Memory

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
000																
010																
020																
030																
040																
050																

Assignment 2

Computer Architecture

```
.globl main
main:
```

```

la s0,b1
lb t0,1(s0) # load byte
lbu t1,0b000001(s0)

lb t2,0x40(s0)
sb t1,0(s0)

la s1, h1
lh t0,0(s0)
lh t1,2(s0)
lh t2,4(s0)
lhu t3,4(s0)
lh t4,20(s0)

la s1, h1
lb t0,0(s0) #check values
lb t1,1(s0)
lb t2,2(s0)
lb t3,3(s0)
lw t4,12(s0)

lui t5,0xabcdf
addi t5,zero,0x723
sw t5,0(s0)

lui t5,0xabcd
addi t5,zero,-1
sw t5,4(s0)

li t0,0xabcd123
sw t0,8(s0)

li t0,0xabcd823
sw t0,12(s0)
```

s0	
t0	
t1	
t2	
t1	
s1	
t0	
t1	
t2	
t3	
t4	
s1	
t0	
t1	
t2	
t3	
t4	
t5	
t5	
t5	
t5	
t5	
t5	
t0	
t0	
t0	
t0	

Write a brief explanatory note analyzing the RISC-V code provided above.

Assignment 2
Computer Architecture

Question 5

Consider the following **RISC-V** code and analyze the working of the **load and shift** instructions. For each instruction, update the contents of all affected registers in both hexadecimal. You are required to clearly understand and explain the execution of every instruction step by step. Also fill memory for given declaration and comment statement if any errors are found

.data memory starts at **0x02000000**

b1: **.byte** 1,2,3,4,5
endbyte1:

.align 2

h1: **.half** 1,2,3,4
endhalf1:

.align 4

w1: **.word** 1,2,3,5,6,7,8,9
endw:

str1: **.asciz** "A very long string"

Memory

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
000																
010																
020																
030																
040																
050																

Assignment 2

Computer Architecture

```
.globl main
```

```
main:
```

```
    la t0, b1
    la t1, endbyte1
    sub t2, t1, t0  # total bytes
```

```
    la t0, h1
    la t1, endhalf1
    sub t2, t1, t0  # total bytes
    srli t3, t2, 2
```

```
    la t0, w1
    la t1, str1
    sub t2, t1, t0  # total bytes
```

```
    li t3, 4
    div t4, t2, t3  # length
    la t0, endw
```

```
    li t1, 0
```

```
loop:
```

```
    lb t2, 0(t0)
```

```
    beq t2, zero, done
```

```
    addi t1, t1, 1
```

```
    addi t0, t0, 1
```

```
j loop
```

```
done:
```

```
    li a7, 10
```

```
    ecall
```

t0	
t1	
t2	

t0	
t1	
t2	
t3	

t0	
t1	
t2	

t3	
t4	
t0	

t1	
----	--

t2	
----	--

t2	
----	--

t1	
----	--

t0	
----	--

Write a brief explanatory note analyzing the RISC-V code provided above.

Assignment 2

Computer Architecture

Question 6

Consider the following **RISC-V** code and analyze the working of the **logical** instructions. For each instruction, update the contents of all affected registers in both hexadecimal. You are required to clearly understand and explain the execution of every instruction step by step

main:

```
li s1, 0x46A1F1B7
li s2, 0xFFFF0000
li a0, 0xFF1c10E7
```

```
and s3, s1, s2
and s4, s2, s1
andi s5, s1, -525
```

```
or s3, s1, s2
or s4, s2, s1
ori s5, s1, -1013
```

```
xor s3, s1, s2
xor s4, s2, s1
xori s5, s1, -923
```

```
not t0, s1
not t1, s2
```

```
slli s3, a0, 7
srli s4, a0, 17
srai s5, a0, 3
```

```
addi t0, zero, 5
sll s3, a0, t0
addi t0, zero, 17
srl s4, a0, t0
addi t0, zero, 93
sra s5, a0, t0
```

```
li a7, 10
ecall
```

s1	
s2	
a0	
s3	
s4	
s5	
s3	
s4	
s5	
s3	
s4	
s5	
t0	
t1	
s3	
s4	
s5	
t0	
s3	
t0	
s4	
t0	
s5	

Assignment 2

Computer Architecture

Question 7

Consider the following **RISC-V** code and analyze the working of the **branch**. You are required to clearly understand and explain the execution of every instruction step by step

main:

```
li s1,0x46A1F1B7
li s2,0xFFFF0000
li a0,0xFF1c10E7
```

```
and s3,s1,s2
and s4,s2,s1
andi s5,s1,-525
```

```
or s3,s1,s2
or s4,s2,s1
ori s5,s1,-1013
```

```
xor s3,s1,s2
xor s4,s2,s1
xori s5,s1,-923
```

```
not t0,s1
not t1,s2
```

```
slli s3,a0,7
srli s4,a0,17
srai s5,a0,3
```

```
addi t0,zero,5
sll s3,a0,t0
addi t0,zero,17
srl s4,a0,t0
addi t0,zero,93
sra s5,a0,t0
```

```
li a7, 10
ecall
```

s1	
s2	
a0	
s3	
s4	
s5	
s3	
s4	
s5	
s3	
s4	
s5	
t0	
t1	
s3	
s4	
s5	
t0	
s3	
t0	
s4	
to	
s5	

Assignment 2

Computer Architecture

Question 8

Examine the following **RISC-V** code and analyze how the **branch** instruction operates. Determine whether the branch will be **taken or not taken**, based on the given register values and conditions.

```
addi s0,zero,-5
addi s1,zero,8
blt s0,s1,label1
addi s3,zero,1
label1:
addi s3,zero,2
```

```
addi s0,zero,-5
addi s1,zero,8
bltu s0,s1,label2
addi s3,zero,3
label2:
addi s3,zero,4
```

```
addi s0,zero,-12
addi s1,zero,10
bge s0,s1,label3
addi s1,zero,5
label3:
addi s3,zero,6
```

```
addi s0,zero,-12
addi s1,zero,10
bgeu s0,s1,label4
addi s1,zero,7
label4:
addi s3,zero,8
```

TAKEN

NOT TAKEN

Assignment 2
Computer Architecture

Question 9

Given the following **C code**, write the equivalent **RISC-V** assembly code.

C Code

```
int main()
{
    int a = 10;
    int b = 20;
    int c = 5;

    if ((a < b && b > c) || (a == c))
    {
        cout<<"Condition is true\n";
    }
    else
    {
        cout<<"Condition is false\n";
    }
    return 0;
}
```

RISC- V Code:

Assignment 2
Computer Architecture

Question 10

Given the following **C code**, write the equivalent **RISC-V** assembly code.

C Code

```
int i = 1, j;  
for (i = 1; i <= 3; i++) {  
    j = 1;  
    cout<<"Outer loop iteration";  
  
    while (j <= 2)  
    {  
        cout<<j;  
        j++;  
    }  
}
```

RISC- V Code:

Assignment 2
Computer Architecture

Question 11

Given the following **C code**, write the equivalent **RISC-V** assembly code.

C Code

```
int main() {
    int arr[10] = {12, -5, 23, 7, -18, 34, 0, -2, 15, 9};

    int smallest = arr[0];
    int largest = arr[0];

    for (int i = 1; i < 10; i++) {
        if (arr[i] < smallest) {
            smallest = arr[i];
        }

        if (arr[i] > largest) {
            largest = arr[i];
        }
    }
}
```

RISC- V Code:

Assignment 2

Computer Architecture

Question 12

Consider the following **RISC-V** program. For each iteration, record the changes made to memory (in rough work) and attach it with your submission. After every iteration, the updated memory values should be shown in decimal form. You are required to carefully analyze the program and write a short explanatory note describing its working.

.data memory starts at 0x02000000

arr: **.word** 12, -5, 23, 7, -18, 34, 0, -2, 15, 9
swap: **.word** 0

.globl main

main:

```
la t0, arr
la t1, swap
lw t5,0(t1)
sub t2, t1, t0
addi t3,zero, 4
div t4, t2, t3
```

start:

```
la t0,arr
add t5,zero,zero
```

loop1:

```
lw s0,0(t0)
lw s1,4(t0)
bgt s0,s1, noswap

sw s1,0(t0)
sw s0,4(t0)
addi t5,zero,1            #swap
```

noswap:

```
addi t0,t0,4
add t2,t1,zero
addi t2,t2,-4
blt t0,t2,loop1
```

```
bne t5,zero,start
```

```
li a7, 10
ecall
```

Short Note:

Assignment 2

Computer Architecture

Question 13

Consider the following **RISC-V** program. For each iteration, record the changes made to memory and stack (in rough work) and attach it with your submission. After every iteration, the updated memory values should be shown in decimal form. You are required to carefully analyze the program and write a short explanatory note describing its working.

```
.data    memory starts at 0x02000000
```

```
arr:  .word 12, -5, 23, 7, -18, 34, 0, -2, 15, 9
swap: .word 0
```

```
.globl main
```

```
main:
```

```
main:
```

```
    la a0, arr
    la a2, swap
```

```
    sub a1,a2,a0
    jal ra,funct
```

```
li a7, 10
```

```
ecall
```

```
##### function#####
```

```
funct:
```

```
    sw zero,0(a2)
    mv t0,a0
    lw t5,0(a2)
```

```
loop1:
```

```
    lw t3,0(t0)
    lw t4,4(t0)
    blt t3,t4, noswap
```

```
    sw t4,0(t0)
    sw t3,4(t0)
    addi t5,zero,1
    lw t5,0(a0)
```

```
noswap:
```

```
    addi t0,t0,4
    addi t2,a2,-4
    blt t0,t2,loop1
    bne t5,zero,funct
```

```
jr ra
```

Short Note:

Assignment 2

Computer Architecture

Question 14

Consider the following **RISC-V** program. For each iteration, record the changes made to memory and stack (in rough work) and attach it with your submission. After every iteration, the updated memory values should be shown in decimal form. You are required to carefully analyze the program and write a short explanatory note describing its working. Look at both function calls saving all registers

.data memory starts at 0x02000000

arr: **.word** 12, -5, 23, 7, -18, 34, 0, -2, 15, 9

swap: **.word** 0

<pre> .globl main main: la a0, arr la a1, swap jal ra, funct li a7, 10 ecall #####function##### funct: addi sp, sp, -12 sw ra, 0(sp) sw a0, 4(sp) sw a1, 8(sp) sw zero, 0(a1) mv t0, a0 loop1: lw t1, 0(t0) lw t2, 4(t0) bgtu t1, t2, noswap addi sp, sp, -28 sw ra, 0(sp) sw t0, 4(sp) sw t1, 8(sp) sw t2, 12(sp) sw t3, 16(sp) sw a0, 20(sp) sw a1, 24(sp) mv a0, t0 jal ra, swapftn </pre>	<pre> lw ra, 0(sp) lw t0, 4(sp) lw t1, 8(sp) lw t2, 12(sp) lw t3, 16(sp) lw a0, 20(sp) lw a1, 24(sp) addi sp, sp, 28 noswap: addi t0, t0, 4 addi t1, a1, -4 blt t0, t1, loop1 lw t3, 0(a1) beq t3, zero, skip jal ra, funct skip: lw ra, 0(sp) lw a0, 4(sp) lw a1, 8(sp) addi sp, sp, 12 jr ra #####swap ftn##### swapftn: lw t1, 0(a0) lw t2, 4(a0) sw t1, 4(a0) sw t2, 0(a0) addi t3, zero, 1 sw t3, 0(a1) jr ra </pre>
--	---

Assignment 2

Computer Architecture

Short Note:

Question 15

Consider the following **RISC-V** program. For each iteration, record the changes made to memory and stack (in rough work) and attach it with your submission. After every iteration, the updated memory values should be shown in decimal form. You are required to carefully analyze the program and write a short explanatory note describing its working. Look at both function calls saving all registers

.data memory starts at 0x02000000

arr: **.word** 12, -5, 23, 7, -18, 34, 0, -2, 15, 9

swap: **.word** 0

.globl main

main:

la a0, arr

la a1, swap

jal ra, funct

li a7, 10

ecall

#####function#####

funct:

addi sp, sp, -12

sw ra, 0(sp)

sw a0, 4(sp)

sw a1, 8(sp)

sw zero, 0(a1)

mv t0, a0

loop1:

lw t1, 0(t0)

lw t2, 4(t0)

bgt t1, t2, noswap

sw t2, 0(t0)

sw t1, 4(t0)

addi t3, zero, 1

sw t3, 0(a1)

noswap:

addi t0, t0, 4

addi t1, a1, -4

blt t0, t1, loop1

lw t3, 0(a1)

beq t3, zero, skip

jal ra, funct

skip:

lw ra, 0(sp)

lw a0, 4(sp)

lw a1, 8(sp)

addi sp, sp, 12

jr ra

Short Note:

Assignment 2
Computer Architecture

Question 16

Given the following **C code**, write the equivalent **RISC-V** assembly code.

C Code

```
#include <stdio.h>

void findMinMax(int arr[], int size, int index, int *smallest,
int *largest)
{
    if (index == size)
        return;

    if (arr[index] < *smallest)
        *smallest = arr[index];

    if (arr[index] > *largest)
        *largest = arr[index];
    findMinMax(arr, size, index + 1, smallest, largest);
}

int main() {
    int arr[10] = {12, -5, 23, 7, -18, 34, 0, -2, 15, 9};
    int smallest = arr[0], largest = arr[0];

    findMinMax(arr, 10, 1, &smallest, &largest);

    printf("Smallest element: %d\n", smallest);
    printf("Largest element: %d\n", largest);
    return 0;}
```

RISC- V Code:

Assignment 2
Computer Architecture

Question 17

Consider the following **RISC-V** code and analyze the working of the **slt** instructions. For each instruction, update the contents of all affected registers in both hexadecimal and decimal format. You are required to clearly understand and explain the execution of every instruction step by step.

addi s0,zero,-5	s0	
addi s1,zero,7	s1	
slt t0,s0,s1	t0	
slti t1,s0,2	t1	
slt t2,s0,s1	t2	

Question 18

Consider the following **RISC-V** code and analyze the working of the **slt** instructions. Write equivalent C code.

RISC-V	C- CODE
addi s1, zero, 0 addi s0, zero, 0 addi t0, zero, 10 for: slt t2, s0,t0 beq t2,zero,done add s1, s1, s0 slli s0, s0, 1 j for done:	

Assignment 2
Computer Architecture

Question 19

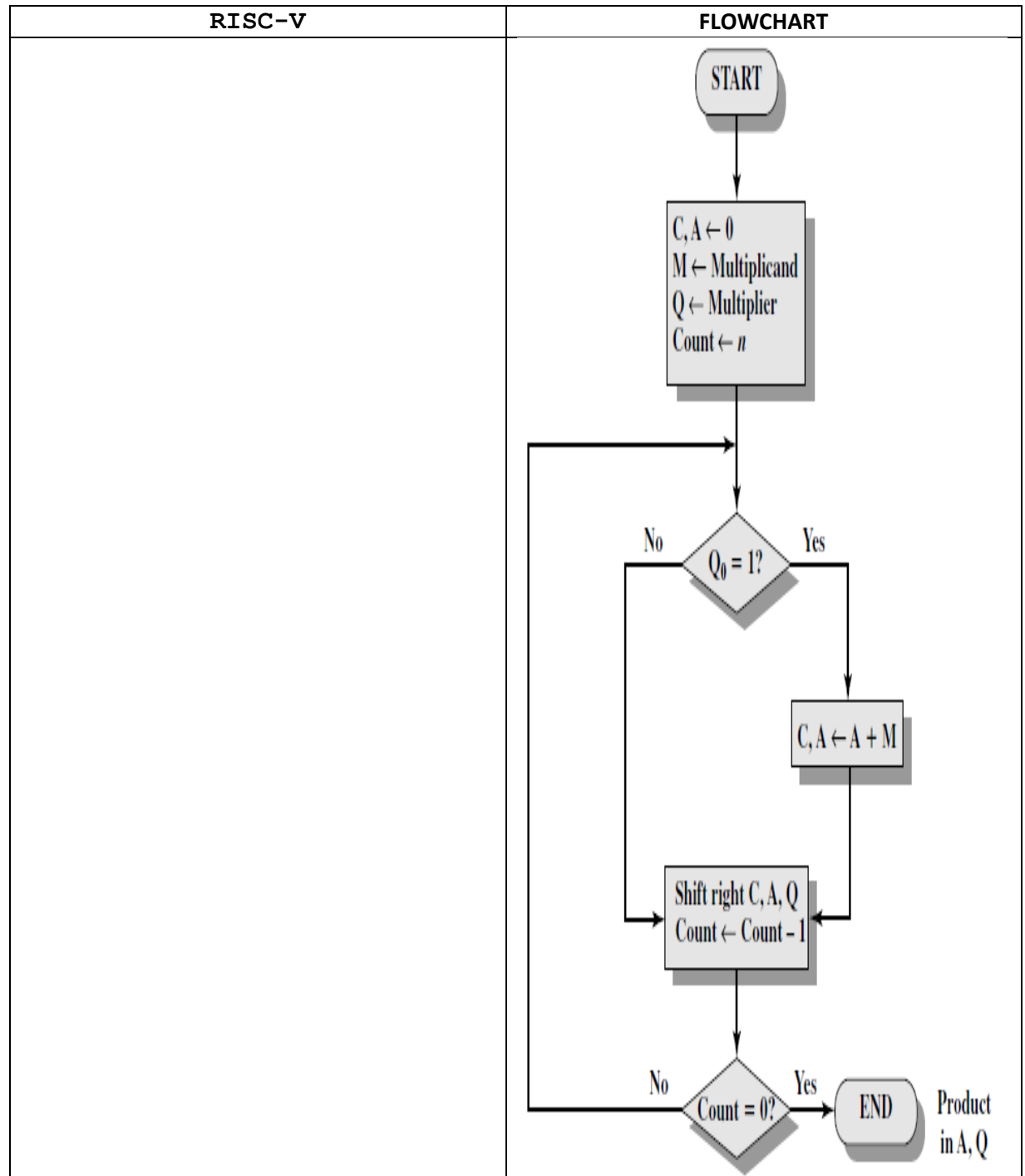
Consider the following **RISC-V** code and analyze the working of the **slt** instructions. Write equivalent C code.

RISC-V	C- CODE
<pre>#s0=base address of char array addi s1, zero, 0 addi t3, zero, 10 for: bge s1, t3, done add t4, s0, s1 lb t5, 0(t4) addi t5, t5, -32 sb t5, 0(t4) addi s1, s1, 1 j for done:</pre>	

Assignment 2
Computer Architecture

Question 20

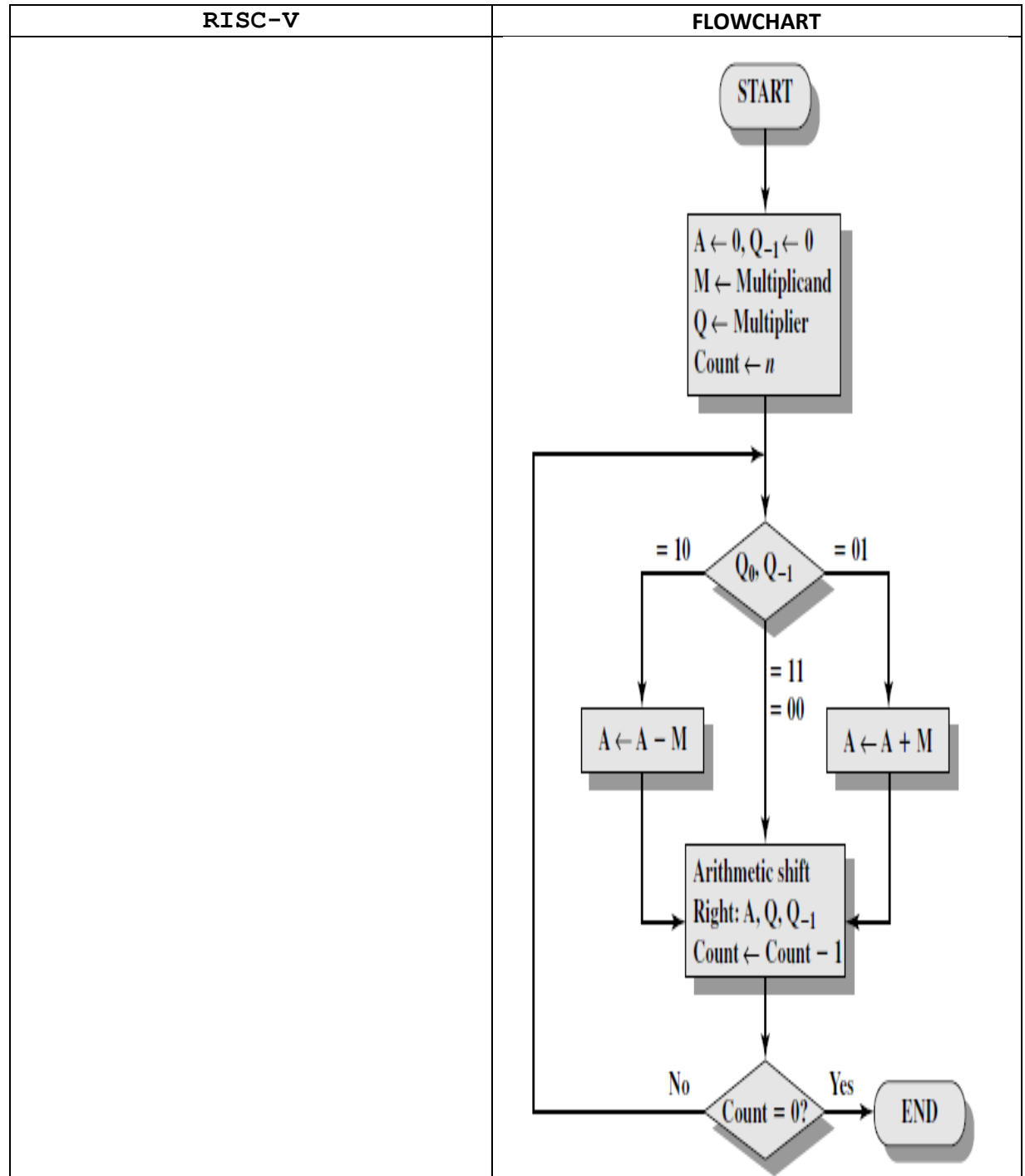
Consider the following flow chat and write **RISC-V** code



Assignment 2
Computer Architecture

Question 21

Consider the following flow chat and write **RISC-V** code



Assignment 2
Computer Architecture

Question 21

Consider the following flow chat and write **RISC-V** code

